School of Electronic Engineering and Computer Science

Final Year Report

Programme of study:
BSc Computer Science and Mathematics with Industrial Experience

Project Title:
Voice Based Email System for the Blind

Supervisor:
Paulo Olivia

Student Name:
Nabilah Joarder

Final Year
Undergraduate Project 2019/20

Date: 10/05/2020

Disclaimer

This report, with any accompanying documentation and/or implementation is submitted as part requirement for the degree of BSc Computer Science with Industrial Experience at Queen Mary University of London. It is the product of my own labour except where indicated in the text. The report may be freely copied and distributed provided the source is acknowledged.

Acknowledgements

I would like to thank the following people, without whom I would not have been able to complete the implementation of this project.

I would like to thank my supervisor Paulo Olivia for his continued support, motivation and guidance throughout the completion of the project.

I would also like to thank Tony Stockman (senior lecturer at Queen Mary University of London) for his assistance in helping me gather my project requirements and providing me with feedback.

My fellow colleagues at university for providing me continuous feedback and advice.

Finally, a huge thanks to my friends and family for all the unconditional love and support in this very intense academic year.

Abstract

The passing of information, ideas or thought from one end to another is known as communication. The most common form of electronic communication today is Emails due to it being the cheapest and most reliable way of communicating. However, not everyone can access email with ease. A huge population of visually impaired users face usability and accessibility challenges when it comes to sending and receiving emails.

This report describes a voice based email system for the visually impaired. A solution that can make it easier and more efficient for visually impaired users to access and make use of all the email functionalities. It describes the way in which this was achieved by gathering requirements, designing and implementing the application and concluding whether the application developed fulfils the objectives set out for this project.

Table of Contents

List of Tables

Table of Figures

Chapter 1: **Introduction**

1.1 Background

We are at the edge of a technological revolution that will essentially change the way we live, work and communicate with one another. This is known as the fourth industrial revolution. Communication is a core factor to society's foundation. It helps to simplify and facilitate the process of sharing information and knowledge with others in conjunction with helping people establish key relationships. The development of technology such as the internet, text messaging and smartphones have widely transformed the way in which people communicate.

One principle method of online communication is email. Email is a fast and reliable means of effective communication. It has many uses such as overcoming barriers, providing wide access, facilitating face to face contact and providing a service. Nowadays it is essential for everyone to have an email address. However, to compose and read emails for the visually impaired can be challenging as all activity performed on computers is based on visual perception. Accessibility serves as one of the founding pillars of user experience and design (Mistry, 2019). Many email marketers do not take into consideration the population of those with visual disabilities and thus make reading emails a challenge.

Globally, at least 2.2 billion people have a visual impairment or blindness (Who.int, 2019). Some visually challenged people may not be computer literate thus, will find it difficult to use screen readers that require keyboard input. Therefore, a multimodal system that uses interactive voice response without the use of a keyboard is crucial to make email communication efficient and seamless to the visually impaired user.

1.2 Problem Statement

In current existing systems in order to send and receive emails, users are required to be keyboard literate. This may prove a challenge for users that have become visually impaired later on in their life as they are required to learn keyboard shortcuts and get training on new systems. Such systems focus mainly on text to speech but do not take into consideration speech to text. A major issue with these systems is the fact that there is no form of interactive voice response or audio feedback which provide users the ability to control the application using only their voice. Although the main target audience are visually impaired users, the application is also aimed at being accessible by everyone. Hence, with the goal of being multimodal.

1.3 Aim

This project aims to help navigate visually impaired users through the Gmail email client using their voice. Users are able to compose and listen to emails through technologies such as speech to text, text to speech and interactive voice response. Moreover, this overcomes the disadvantages of the existing system in that it is fully voice-based as the system works on vocal abilities, reducing the software load of using screen readers and user's cognitive load on remembering keyboard shortcuts.

1.4 Objectives

In order to achieve the aim of this project, the following objectives must be reached:

- To compare and contrast existing models and establish key features and limitations of those systems
- To gain an understanding of relevant concepts such as text to speech, speech to text and interactive voice response.
- Data will be collected from visually impaired users by conducting interviews on ways in which email is currently used before implementation.
- To identify and treat business risks.
- To implement a user interface that can be easily accessible by the text to speech system
- To create and develop a system that will be accessible to visually impaired users using interactive voice response.
- To ensure more productivity for users by incorporating speech to text technology.
- To comply with rules, regulations, standards and practices.
- To test out the application after completion
- To produce an application that allows users to navigate through the Gmail email client performing functions such as composing email using only voice.

1.5 Research Questions

The questions this project is set to answer is highlighted below:

- How can an application that sends and receives emails to users without the use of a keyboard be achieved?

- How can you improve the experience of email communication for visually impaired users?

- Does disregarding the use of a keyboard allow an easier way to achieve tasks for the visually impaired?

- Can a multimodal email system be beneficial to all users?

Chapter 2: **Literature Review**

Email is an essential component for collaboration and communication; however, computer software, websites and applications can demonstrate accessibility and usability barriers for visually impaired users. In particular, according to Borodin (2008), technology in web-based applications such as Flash and AJAX can cause many issues. This chapter provides a detailed analysis of the impact of assistive technology and provides information on existing systems in place in order to gather requirements for the proposed solution.

2.1 Assistive technology

Assistive technology in computer science is computer hardware and/or software that is used to increase, maintain, or improve the functional capabilities of individuals with disabilities. It promotes independence, choice, control and enablement with the sole aim of enabling individuals to continue to live independently and improve their quality of life. To ensure access to all potential users, it is important that software producers avoid creating access barriers to people with disabilities and develop products that are compatible with assistive technology (Burgstahler, 2014):

Being able to take all users into consideration and designing a product that can be used for all users is called “universal design”. According to Golden (2001), in a survey of 25 award winning companies who produce instructional software, 65% of the companies were not aware of accessibility as an issue. This determines the need for making software accessible to all individuals with or without disabilities. As stated by Judd-Wall (1999), it is crucial to note the levels in applying the assistive technology solution which includes whether the item is personally, developmentally, or instructionally necessary for there to be a need and it to be the most effective.

In particular, according to Mack, Koenig, and Ashcroft (1990), for educational success students who are visually impaired or blind need to be proficient in using access technology. Thus, assistive technology produced should be designed in a way that is straightforward, easy and efficient for the target audience.

2.2 Similar Systems

2.2.1 The use of Screen Readers and Email

According to the World Health Organisation, the estimated number of people visually impaired in the world is 285 million, 39 million blind, and 246 million having low vision. (WHO, 2010). A screen reader is a key form of assistive technology that helps enable those who are visually impaired to use a computer.

A screen reader works by delivering information to the user via speech or braille. It uses a Text to Speech engine built into the screen reader that

translates information on screen into speech audio. As well as speech output, screen readers can also provide users information in braille with the use of an external hardware device incorporated with speech output (Watson, 2005).

With the use of screen readers and the keyboard visually impaired users are able to read and send emails by voice feedback and keyboard shortcuts. The two main screen readers on the market are JAWS and NVDA (non-visual desktop access). The JAWS screen reader has a large community of users with many useful features that provide access across multiple platforms. It is known to be the most powerful screen reader on the market. However even though it is the most powerful it is also the most expensive with the retail price being around £838. Every time the system updates users are required to pay a fee to upgrade to the latest version. This is very costly and not all everyday users could afford this.

An alternative to the JAWS screen reader is the NVDA screen reader. This is now becoming more popular and is a big competitor of JAWS. With the cost of this screen reader being free this is a go to option for the everyday user.

To be more specific to email communication, screen readers do present many limitations when it comes to usability. According to research conducted by (Wobbrock, & Ladner, 2007). Blind users are more likely to avoid something when they know that it will cause them accessibility problems. Additionally, application issues that slow down a work task cause the most frustration for blind users (Lazar, Feng, & Allen, 2006).

Wentz and Lazer (2010) conducted a web-based survey on email usability and email problems that could be negatively impacting blind users. According to the survey areas that users would like to see improved includes web-based interfaces, calendaring and email address book. Other areas of concern were with the search functionality, spam emails and email attachments.

Alongside looking at previous case studies, my market research involved a comprehensive search for current existing systems that help aid visually impaired people in sending and receiving emails. The main applications I have found was the voiceover feature on Mac and Microsoft dictate on Outlook. By evaluating and critiquing these existing applications an outline of features that work well on these applications and points that can be improved have been concluded.

2.2.2 Mac Voiceover

Voiceover is a built-in screen reader on mac which can be accessed as follows: Mac > System Preferences > Accessibility > Voiceover. This is presented with large text as some visually impaired users have low vision. This feature describes exactly what is happening on the screen. It gives users descriptions of each on screen element providing useful hints along the way. In addition, Voiceover includes features such as keyboard-based navigation. A useful feature the mac voice over has compared to windows is the ability to alert users when a background application is launched allowing easy navigation. In terms of writing emails voiceover has spell checking and auto correct functions

available system wide as well as an extensive search mechanism (Prater, 2018).

Some of the issues found with voiceover is as follows: in order to send an email on mac using voiceover and apple mail, users would have to type the body of the email using the keyboard. This will be an issue to those who are not keyboard literate. Additionally, Voiceover can have issues identifying attached files when composing emails which is a huge concern for users.

2.2.3 Microsoft Dictate

Dictate is an add on for Microsoft Office on Windows. In particular users can write emails using voice on outlook. Aside from outlook users are able to use the dictate feature on Microsoft Word and PowerPoint. In order to enable dictate on Outlook simply click new email> click dictate > speak. Different languages are supported therefore it is versatile for many users. In order to use the dictate feature on emails users must click new email first or reply to an email then click the dictate button. The ability for speech input is limited to only requiring input when writing the email body. In addition to this there is no audio feedback and limited vocabulary as new industry specific vocabularies are being added (Voxtab, 2015). Moreover, this system has not got an interactive voice response mechanism in place.

2.3 Comparison of Similar Systems

A comparison of key features and limitations identified in existing systems and how it compares to my system is demonstrated in the table below:

Application	Features Identified	Limitations
Screen Reader (JAWS and NVDA)	• Ability to personalise the speed of speech output • Accurate and powerful speech output • Ability to use braille • Announces spelling styling and formatting information.	• Can be expensive • Not able to recognise computer graphics well. • No interactive voice response/ speech input. • Requires frequent updates which cost a fee.
Mac Voice Over	• Gives description of each onscreen element and provides helpful hints • No need for installations as it is already installed on MacBook's. • Notifies the user when an app is frozen.	• Application only available on MacBook's, not available on Windows. • Expensive • Requires the use of a keyboard. • Since it is not an app to update voiceover Apple will have to update OSX as a whole.

Microsoft Dictate	• Provides voice input • Does not require the use of a keyboard • Can capture speed faster than type. • Has helpful punctuation commands	• Does not provide voice output. • Frequent pauses when converting speech to text. • Only can be used when writing the email body. • Only limited to Microsoft
AudioMail (My System)	• Multimodal: Users can navigate through the app with either just using voice or keyboard input or both. • Interactive and provides audio feedback. • Can add and download attachments to emails using voice. • Easy to navigate • Keyboard is not required. • Can perform most email functionalities, e.g. star, forward, move to spam etc. using voice. • Has a simple and unique interface assessable by all users.	• Limited to specific browsers e.g. google chrome and Microsoft edge due to the capabilities of the front-end libraries.

Table 1: Comparison of similar systems cross referencing my application

To conclude, after analysis of several existing systems. Most of which serve the main purpose for visually impaired users in sending and receiving emails with ease. Minor faults and areas of improvement were identified in the majority of the current systems available. One of the main issues identified was the fact that there is no interactive voice response system in place for either screen readers, Mac voice over or Microsoft dictate. The current systems available only have either speech output or speech input but not both. Additionally, most systems require the use of a keyboard which means that users would have to be keyboard literate.

2.4 Proposed Solution

The aim of this project is to develop an application in such a way by taking into consideration the areas to improve from existing applications highlighted in the comparison table above and focus mainly on interactive voice response. Other technologies that will be utilized is Speech to text and text to speech. With the majority of current systems lacking the functionality of voice input the goal is to incorporate the ability to send and receive emails using only voice whilst

eliminating the use of a keyboard. This system will provide visually impaired users the ability to access emails easily and efficiently.

The diagram below illustrates the processes the system will take in order to fulfil the user's requests. To be more specific figure 1 demonstrates the users journey and how the system works.

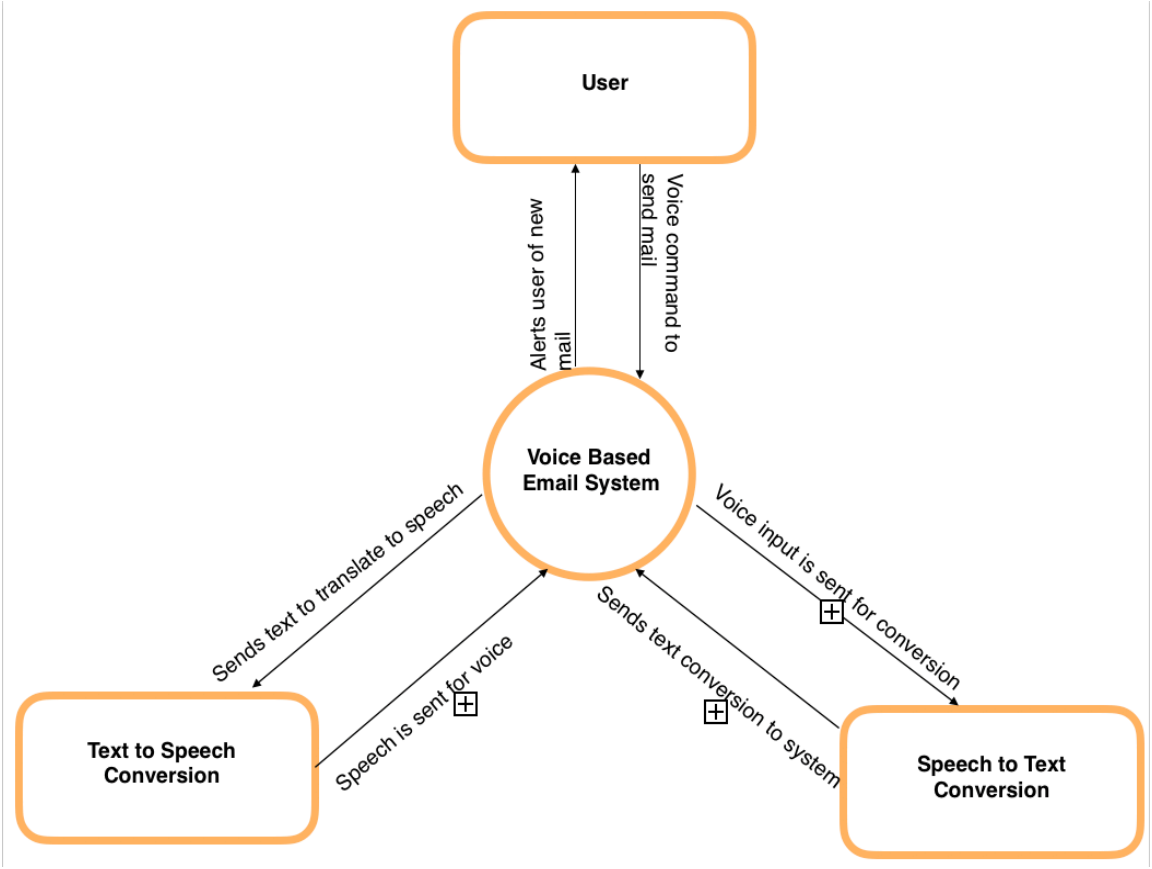

Figure 1: Processes the system takes in order to fulfil user's requests

Chapter 3: **Requirements Analysis**

3.1 Requirements Analysis

In order to gather requirements, analysis of existing systems and case studies were carried out. In addition, an interview was conducted on Tony Stockman, senior lecturer at Queen Mary University of London to gain deeper insight into the targeted users concerns and needs which will then help me design a system that is well suited to needs of the users. Interviews with other users were also carried out in order to calculate an average and gather accurate data for evaluation. This has helped gain understanding in existing systems and shaped the project so that it is different to current systems. The questions asked in the interview are stated below:

1) How do you currently use email?
2) Do you have any difficulties when using email?
3) Can you tell if email is spam?
4) What features would you like to see to help improve your experience?

3.2 Analysis of Results

The first question was based on current existing systems. Getting an idea of what users currently use when reading and sending emails. Response from the interview as well as other sources claim that the JAWS screen reader is the most popular when reading and sending emails with a large community of users. With NVDA coming close second as it is a cheaper option. Screen readers are the main technology that help users who are visually impaired to access and interact with digital content.

The second question focused on limitations of existing systems. This emphasised the importance of specific features that can be improved or added to make user's experience of sending and receiving emails much easier. Responses from the interview highlighted the need for an interactive user experience. Currently, existing systems only have either voice input or voice output but not both. Additionally, another limitation highlighted was the need for screen readers to better recognise computer graphics. For example, in particular the use of CAPTCHA's (Completely Automated Public Turing test to tell Computers and Humans Apart) for authentication. This test enables users to select specific pictures in order to get access to a system.

The third question looked more into the security aspect of things. To be specific spam emails. Spam emails have the potential to cause major harm, annoyance and distress therefore, it is important to know if current systems used by the visually impaired can detect these emails. According to the response received users currently detect spam by reading the subject line and the from address and decide whether it is from a familiar address or not. Most email systems have spam filters switched on which will send the email straight to junk before the user can see it in the inbox.

The final question focused more on what features users would like to see in order to improve their experience with sending and receiving emails. Based on the response received, a feature that would be useful to users is the ability to be notified if an email is of high importance. This will alert the user to pay immediate attention to specific emails. As well as this, the ability to pick out emails from particular users was also of major interest. For example, users would like to see emails coming from there manager or specific colleagues first.

Moreover, the ability for voice input and a decent dictation system without the use of a keyboard was another feature that is of interest to users as not only will visually impaired users benefit from this feature, users with other disabilities such as those who suffer from Phocomelia, a condition that involves malformations of the arms and legs and other conditions which places difficulty in using a keyboard. Additionally, in regard to security another feature that was mentioned was the ability for a better spam email detection and audio authentication rather than CAPTCHA graphic authentication as currently screen readers have difficulty reading graphics due to the text being made up of graphics which cannot be recognised well by screen readers.

In conclusion, interviews were conducted in order to identify key aspects that could be applied to the project. This research was useful as it highlighted key requirements to consider in the application in order to provide a seamless experience for the ideal user.

Based on the response received, research into existing systems and previous case studies, the target is to create an application which focuses on interactive voice response to send and receive emails without the use of a keyboard.

3.3 Project Requirements

Based on previous case studies, my research and responses from the interview, analysis on some hardware and software requirements along with functional and non-functional requirements were carried out. By taking into consideration key features and functionalities users would like to see.

3.3.1 Hardware Requirements

1) **Desktop or laptop with decent amount of CPU and memory**
   For faster performance, a system with a good amount of CPU and memory will be more beneficial for speech recognition as it demands a lot of memory.

2) **WIFI connection**
   In order for the web based system to function a secure WIFI connection is required in order to use the application.

3) **A good microphone**
   A good microphone is required for improved accuracy of words.

4) **Functional speakers**
   Speakers that give a clear output is essential to make to ensure words are understood.

3.3.2 Software Requirements

The table below outlines the software that is necessary to implement the application along with the versions required.

Software	Version
Python	3.7.0
Django	3.0.5
JavaScript	1.8.5
Bootstrap	4.0
BeautifulSoup	4.9.0
PostgreSQL	N/A
Web Speech API	N/A
ResponsiveVoice API	1.6
imaplib	4.0
smptplib	N/A
Cloudinary	1.20.0

Table 2: List of software along with the versions required

3.3.3 Functional Requirements

	Requirement	Completed
1.	Users should be able to register and login into the application using voice input and transform it to text. The system should check the text to see if the user is valid.	√
2.	Once logged in the application should give feedback to the user of the location.	√
3.	The application should display users' emails retrieved from Gmail using the python's IMAP library	√
4.	Users should be able to compose mail with "compose mail" command.	√
5	With voice input users should be able to enter information in the "to", "from", "subject" and body fields of the email which will be translated to text for the user to then say the "send mail" command for the email to be sent.	√
6.	Users should be able to read mail by speech output and know where it is from.	√
7.	Users should be able to delete mail which will then be displayed in the trash folder using voice input.	√
8.	The application should be able to check for attachments in email and notify the user.	√
9.	Application should be able to detect emails with high priority and notify the user.	√
10.	The application should be able to pick up emails from specific users.	√
11.	The application should be able to let the user stop the audio output.	√

Table 3: Functional Requirements

3.3.4 Non-Functional Requirements

	Requirement	Completed
1.	The system should be accessible by the visually impaired.	√
2.	Application should be responsive without the use of a keyboard.	√
3.	Error prevention and handling should be implemented throughout for easier troubleshooting.	√
4.	Use AJAX wherever possible for asynchronous requests	√
5.	Application should have good security and be able to check for spam emails.	√
6.	The system should provide a quick conversion from speech to text without major delays	√
7.	The system should be multimodal: accessible by all users.	√

Table 4: Non Functional Requirements

3.3.5 Security Requirements

	Requirement	Completed
1.	The system should prevent cross site scripting attacks from occurring.	√
2.	The system should enable cross sire request forgery protection	√
3.	The system should facilitate SQL injection protection	√
4.	Users should ensure IMAP is enabled on their Gmail accounts.	√

Table 5: Security Requirements

Chapter 4: **Design and Method**

In this chapter, the user interface and the transition between web pages are explained in detail. This should give some background familiarity on how the application and all its functionalities work. The design of the application is described by taking into consideration Norman's design principles. Additionally, the system architecture is explained as well as the chosen method used for implementation.

4.1 Design Overview

The diagram below illustrates an overview of the application and how it works. It focuses on the flow of the system and provides an outline of the way in which the application will be implemented.

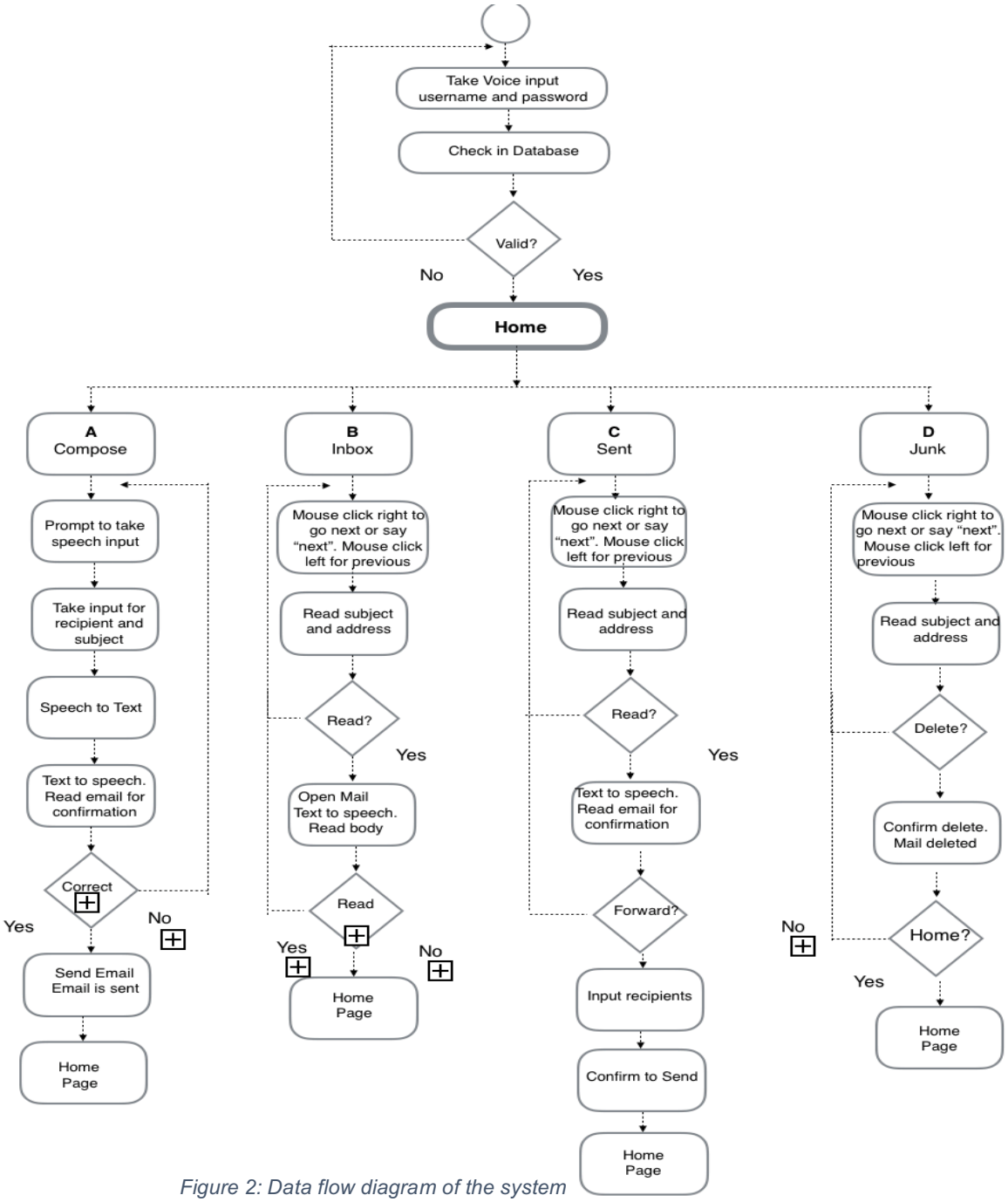

Figure 2: Data flow diagram of the system

4.2 Application Walkthrough

In this section, a brief description of each page of the web application and the way in which the application works will be explained with the aid of screenshots.

The application has two modes:

1. **Traditional Mode:** This mode allows users to navigate their way through the application through clicks and keyboard input i.e. access the application like most sites.
2. **Voice Mode:** This mode allows users to navigate their way through the application via voice input. No keyboard is required.

The application consists of a number of states, to represent the different states and transitions in this application, a state transition diagram was constructed to model the dynamic nature of the system:

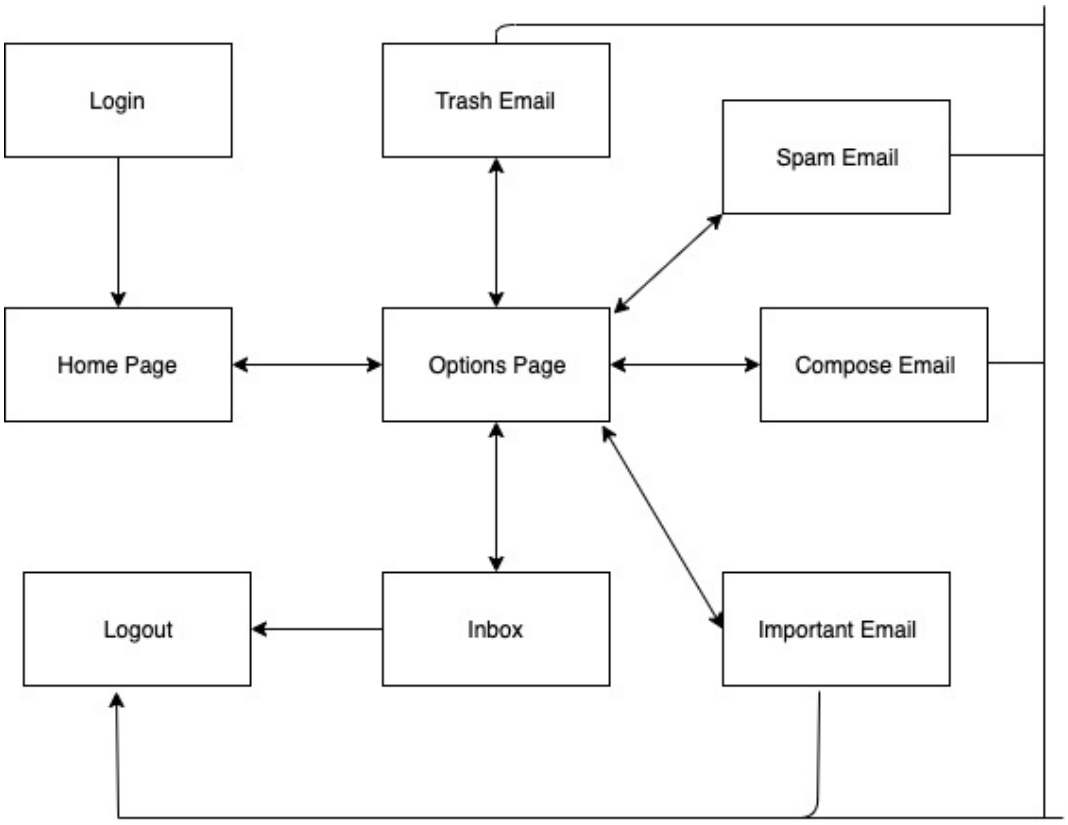

Figure 3: State Transition Diagram

Activating speech:

The application was designed to be multimodal. Users can either use the application with voice or use the application with traditional keyboard input. Due to chrome security restrictions, in order to activate speech, the user must double click on the webpage.

Home page

The homepage of the application is a basic overview of the functionality of the project. Since the main target audience are visually impaired users this homepage is both neutral and simplistic in that it is not only suitable for their needs but also suitable for the needs of others. In order to activate the voice functionality users must double click the page and follow the voice commands.

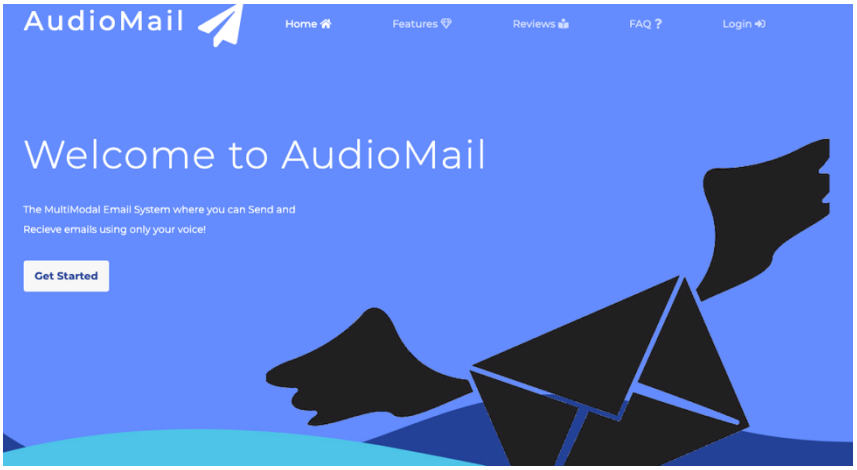

Figure 4: Screenshot of Homepage

Login Page

From the home page, the web application will transition to the login page in which the user will be prompted to enter their Gmail username and password via the voice input. A connection to their Gmail account will be established at this stage.

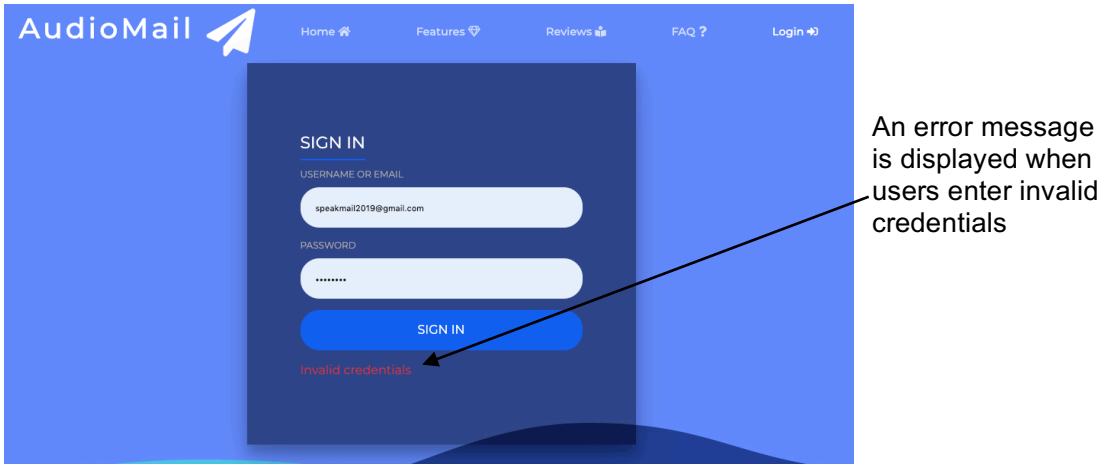

Figure 5: Screenshot of Login page

Options Page

After successfully logging in, if the user chose to log in through voice mode, the user is transitioned to the options page. The user is then presented with the options: “Compose”, “Inbox”, “Sent”, “Important”, “Spam”, “Starred”, “Trash”, “Help” and “Logout”. The user is then prompted to input their desired option via speech. Additionally, if users want to use the application the traditional way a slider option is displayed to switch the application to the traditional mode.

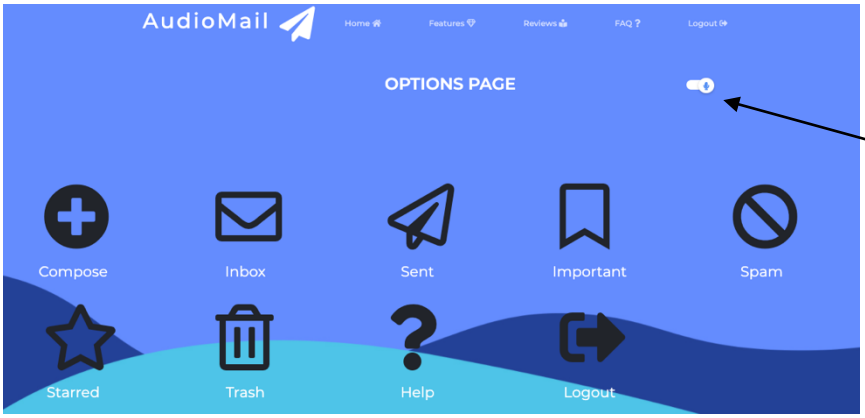

Figure 6 : Screenshot of Options page

Inbox

This page displays the users last 50 emails. Users are able to perform most email functionalities such as mark email as read/unread, star an email, move to different folder, delete email, reply/ forward email and print email. A slider option is also available for users to switch to voice mode. Additionally, users can switch between folders by clicking the dropdown icon. This is shown on Figure 8.

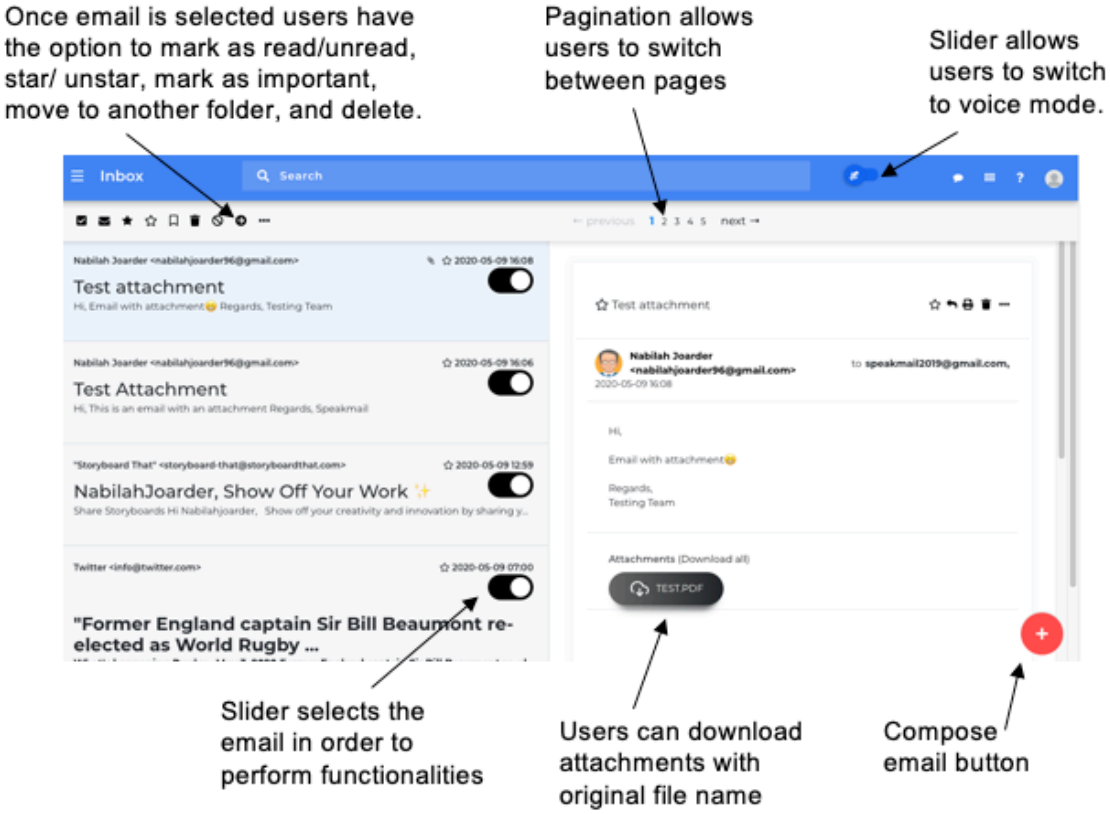

Figure 7 : Screenshot of Inbox page

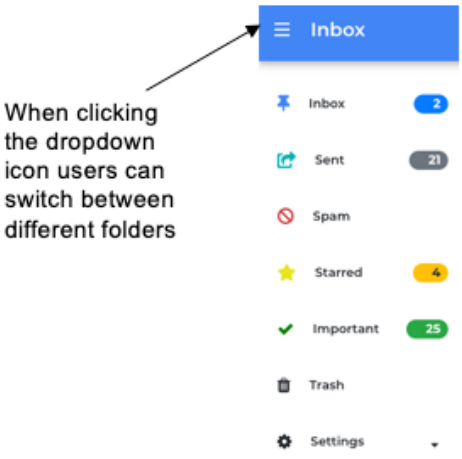

Figure 8 : Screenshot of dropdown on Inbox page

Compose Email

Through voice mode users are prompted to enter the recipients address, subject and email body. Additionally, users also have the option to add an audio attachment. When in traditional mode the compose functionality is shown in Figure 9.

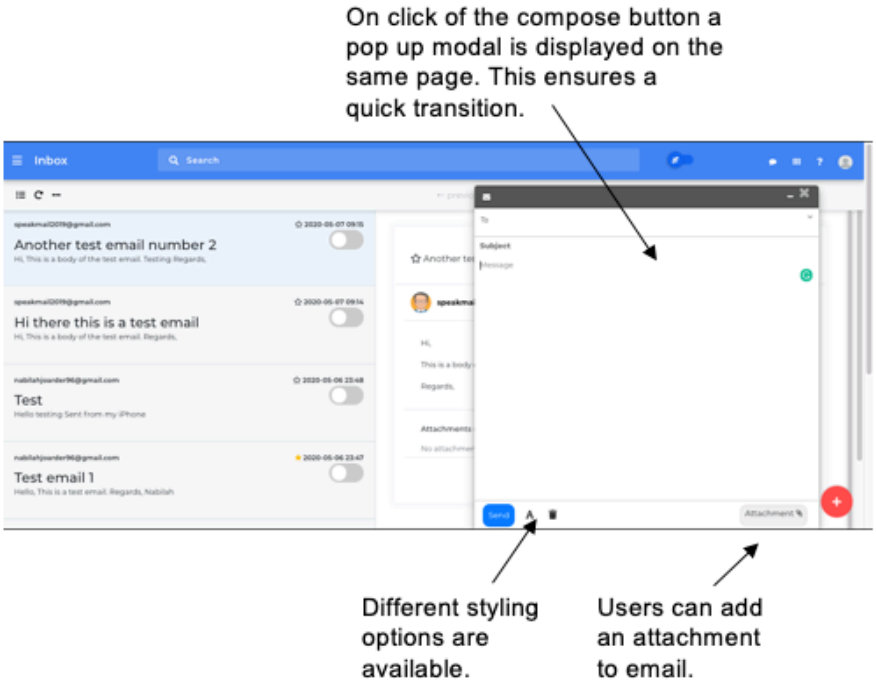

Figure 9 : Screenshot of Compose email functionality

Search Email

On voice mode users are able to search through emails via email address. Once the email is selected users can perform the desired email functionality. On traditional mode users can search for emails by entering a keyword. A list of emails that contain the keyword are displayed as a list. This is shown on Figure 10.

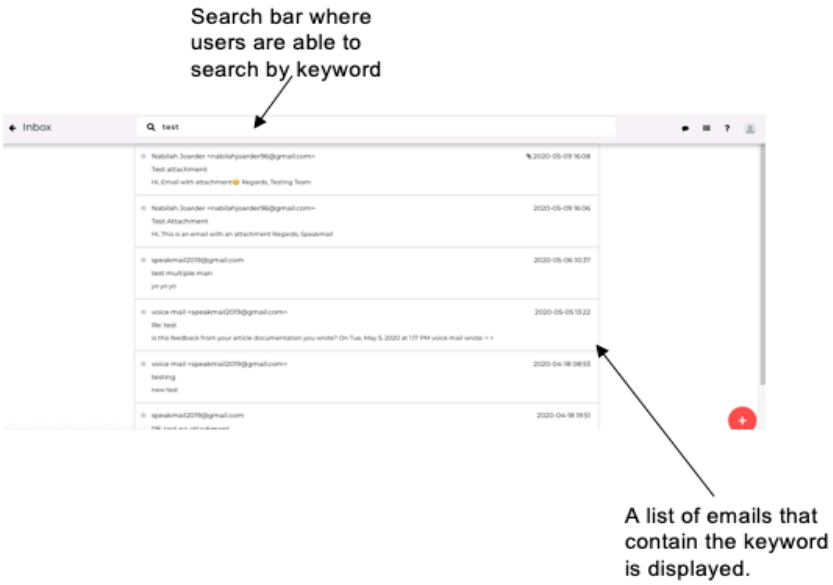

Figure 10: Screenshot of Search functionality

4.3 Norman's Design Principles

In terms of the design of the user interface, the aim was to achieve simplicity and ease of use. This was done by incorporating Norman's design principles. These universal design principles can be applied to design technology products. In particular, the principle of visibility, feedback and affordance was taken into account in order to maximise the user experience of the application.

Since the target audience are visually impaired users, the principle of visibility is applied through processing indicators in speech. For example, the user is notified when to speak by the application which indicates to the user the right time to speak.

The principle of feedback is demonstrated with the interactive aspect of the application, providing feedback on every action and transition. Additionally, feedback is provided by asserting attention. For example, the application provides feedback through audio repeating what the user says to ensure accuracy and providing users with confirmation when an action is performed successfully.

The principle of affordance was considered when creating the options page in the project. It provides users an idea of how to use the application through voice indicating the voice commands that is required to navigate through the application.

Other Norman's design principles were considered when designing the application, a brief summary of how they were used are demonstrated in the table below:

Design Principle	Description	Example
Visibility	The more visible an element is, the more likely users will know about them and how to use them.	Speech indicators indicating suitable time for user to speak.
Feedback	Users should receive confirmation that an action has been performed successfully.	Users receive audio confirmation.
Affordance	Visual attribute of an object or control that gives users an idea of how the object or control is utilised.	The options page gives audio output on instructions.
Mapping	Mapping a relationship between actions in relation to how they work in the real world.	The general structure of an email is mapped the way an email is usually composed.
Constraints	An aspect of design that restricts a user from performing a certain action and preventing invalid data being entered.	Confirmation on audio input is given to user as well as restricting users to log in when session is expired.
Consistency	Having similar operations and similar elements for achieving similar tasks.	In order to perform an action, the user is required to speak after the speech indicator alerts them.

Table 6: Examples of how Norman's Design Principles was implemented.

4.4 Model View Controller Design Pattern

The MVC design pattern was used in order to make it easier to create the application in a more clear and efficient way. This architectural pattern separates an application into three main logical components the model, the view and the controller. Django, takes care of the controller and provides a template. This is illustrated further with the diagram below:

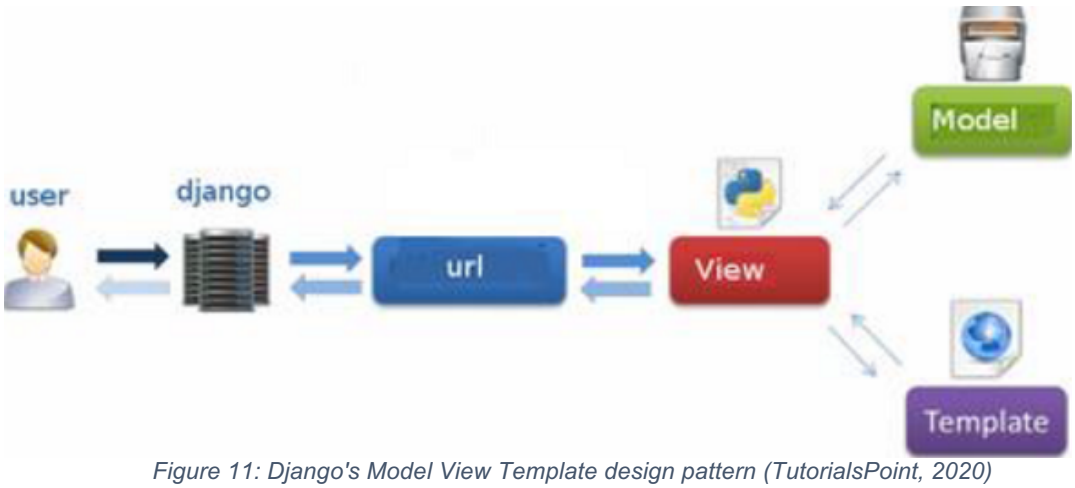

Figure 11: Django's Model View Template design pattern (TutorialsPoint, 2020)

The model represents the underlying, logical structure of data in a software application and updates the view. The view represents the elements in the user interface. The template then maps it to a URL in which Django displays to the user.

4.5 System Architecture

This section details the interaction of the system between client and server. It explains how the client and server communicate in particular what is done on the client side and what is done on the server side.

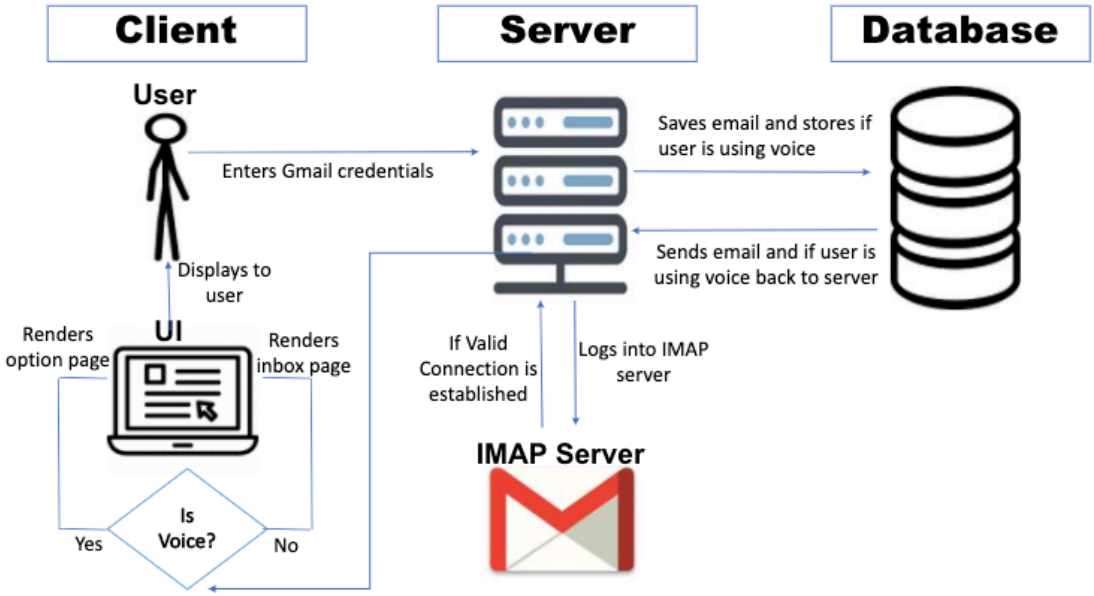

Figure 12 : Diagram of System Architecture

As illustrated in the diagram above, the client enters Gmail credentials which is sent to the server. The server then saves the email and stores whether the user is using voice into the database. The server then retrieves the email and validates if user is using voice and sends credentials to the IMAP server in which it logs the user in and checks if the credentials is valid. If valid, a connection is established and is sent to the server. Moreover, once logged in, the client verifies if user is using voice. If the user is using voice the options page is rendered. Whereas if the user is not using voice the inbox page is rendered and displayed to the user.

4.6 Method

The application was implemented by using an agile method of development, adopting an iterative and incremental design pattern. This breaks down a large software component into smaller chunks thus makes it easier to test and debug. Each time a feature is implemented, it is tested and reviewed in order to identify further requirements. This was achieved by creating a branch for each functionality on Bit Bucket. Once a feature is completed the branch is then merged to the master branch. This method makes adapting to changes much more quick and efficient. Furthermore, it has proven to be beneficial throughout the project. In particular when changing requirements based on user feedback.

Chapter 5: **Implementation**

This chapter provides a thorough description of the technologies used to implement the application as well as highlight the way in which the core functionalities of the application was achieved. Moreover, it outlines the steps taken to achieve successful deployment.

5.1 Languages and Frameworks

This section details the reasoning behind the choice of the languages and frameworks used when implementing the application.

5.1.1 Python with Django

The language used to implement the application is Python, a high-level general-purpose programming language. Python was used for the project due to its versatile features and its large extensive support libraries. Prior to implementing the project, little knowledge of the language was known beforehand. Therefore, with the use of tutorials and documentation, confidence in the language was built up. Django is the web framework used, its written in Python and makes creating applications easier and efficient. The framework is secure and focuses extensively on reducing application development time.

5.1.2 JQuery and Ajax

Ajax loads data in the background and renders it on a webpage without the need for reloading the page. JQuery is a JavaScript library that provides methods for the AJAX functionality so that it can be used on all browsers. Ajax was used in this application in order to reduce traffic between the client and server and increase performance and speed.

5.1.3 Bootstrap

Bootstrap is an open source front end CSS framework that is used for the creation of websites and web applications. It is built on CSS, JavaScript and HTML to facilitate the development of applications. This framework was used due it being lightweight and customizable as well as having responsive structures and styles.

5.2 Essential API's, Libraries and Services

This section outlines a brief description of the required API's, libraries and services needed to develop the application.

5.2.1 Web Speech API

The Web Speech API works in the front end and provides the functionality of speech recognition and speech synthesis. Speech recognition is used in the application to covert speech to text. This involves receiving speech through a device's microphone which is then verified by a speech recognition service against a list of vocabulary and grammar. Once a word is effectively identified, it is returned as a result. In order to initiate the API, the prefixed interface webkitSpeechRecognition is required. The property `recognition.continuous` controls whether continuous results are captured

or a single result every time the recognition starts. Additionally `recognition.interimResults` confirms whether the application should return interim or final results. Furthermore, to initiate the speech to text conversion the method `recognition.start()` is called and speech is captured and converted to text. This is demonstrated on the code snippet below:

```
var recognition = new webkitSpeechRecognition();
recognition.continuous = false;
recognition.interimResults = false;
recognition.lang = "en-US";
recognition.start();
recognition.onresult = function (e) {
text = convert_special_character(e.results[0][0].transcript);
```

Figure 13 : Code to implement speech to text

5.2.2 ResponsiveVoice API

The ResponsiveVoice API is a JavaScript API that converts text to speech. It is created solely for the use of developers to add voice features to their application. The API generates speech in real time on the client side from the browser. To make use of the of the API the following script is called through a CDN (Content Delivery Network) with a unique API key:

```
<script
src="https://code.responsivevoice.org/responsivevoice.js?ke
y=YOUR_UNIQUE_KEY"></script>
```

In order to make the text to speech conversion the following method is called: `responsiveVoice.speak(String text, [String voice], [Object parameters])`

Where `String text` is the text is the text to be spoken, `String voice` is the chosen voice of the speech and `Object parameters` are used to define an optional pitch, rate and volume.

5.2.3 imaplib

The IMAP library implements a client communicating with Internet Message Access Protocol (IMAP). It is an email retrieval protocol that reads and displays emails. Pythons imaplib is the client side library that was used to access emails over an IMAP protocol. Its core job is to map common IMAP commands to python methods. IMAP does not download emails on the local computer it allows the client to manipulate, hold and maintain the email message on the server. The library allows users to perform most email functionalities such as compose, delete download and search for mail.

5.2.4 smtplib

SMTP (Simple Mail Transfer Protocol) is a protocol that operates over TCP/IP which manages sending emails and routing them between email servers. Pythons smtplib library was used to send emails through the GMAIL email server. It defines an SMTP client session object that can be used to send mail to any internet machine with an SMTP listener daemon (Tutorialspoint, 2020).

5.2.5 Amazon Polly

Amazon Polly is a service that converts text to speech. It is used in the application to allow users to send an audio attachment. The service allows speech to be created in mp3 format and served from either the cloud, locally or offline.

5.3 Development of Core Functionalities

In this section, the core implementation of the backend is explained with the support of code snippets.

5.3.1 Login

In order to access emails users must be able to login into the application. This is a fundamental requirement within the project itself. Establishing a connection with the email server is essential. In particular a connection to Gmail is required. The details needed to login are the username and password to the users Gmail account. This is done in the front end via the Web Speech API and ResponsiveVoice API respectively within the file login.py. The connection to the server itself is done via pythons imaplib library in the file views.py which allows the end user to view and manipulate the messages as though they were stored locally on the end users computing device. The connection to the server is established as follows:

```
email = request.POST['emailvoice']
password = request.POST['passwordvoice']

imap_url = 'imap.gmail.com'
conn = imaplib.IMAP4_SSL(imap_url)
try:
     conn.login(email, password)
```

Figure 14 : Code to Login through IMAP

Once the user enters email id and password through voice, a connection is made with the host over an SSL encrypted socket. This ensures all data passed between the web server and browser remains private. The standard port for IMAP4_SSL is 993. The host url is the address of the email server in this case ‘imap.gmail.com’ since Gmail was the chosen server. A connection to the server is then attempted. If successful the application would give audio feedback to notify the user of the successful login. However, if unsuccessful the application

would notify the user on its failed attempt to log in and will prompt the user to enter the required details again.

After successfully logging into the application, the browser stores a session and cookie for that user. A cookie is a small text file stored on the user's computer designed to hold specific data to a particular website. Thus, if a user does not log out and navigates to another website and then redirects back to the original application then that user's session will still remain active.

5.3.2 Receiving Email

In order to fetch emails, Pythons imaplib is the module used to store email messages on a mail server as a means to allow the end user to view and manipulate messages as though they were stored locally on end users computing device.

Once logged in with the Gmail username and password and a connection with the host over an SSL encrypted socket and server is made. In order to retrieve and have full access to emails the function `.select("INBOX")` selects the folder you want to read emails from. In this case the inbox folder is chosen. To search for mail the function `.Search()` searches for mail. Filters can be provided to search for specific mail for example, from, to or subject of mail to be found. To get all mail the function `.search(None, "ALL")` returns all messages without filter with two values one of which is the result of the request for example if the request was successful or not, while the other is the data which is the ids of all the emails. Due to performance, the application retrieves the last 50 emails. Figure X demonstrates an example of this.

```
conn.select('"INBOX"')
result1, data1 = conn.search(None, "ALL")
all_list = data1[0].split()[-50:]
mess_list = read_email_filter(all_list,'"INBOX"')
```

Figure 15: Code to select folder and retrieve emails

To retrieve and display the "to", "from" and "subject" of the mail, iteration from all the id's is required. In order to fetch an email for a given id the function `fetch()` is used where `'RFC822'` is an Internet Message Access protocol. An example of this is shown on figure X:

```
for item in mail_list:
    result, email_data = conn.fetch(item, '(RFC822)' )
    raw_email = email_data[0][1]
    message = email.message_from_bytes(raw_email)
    To = message['To']
    From = message['From']
    Subject = message['Subject']
```

Figure 16 : Code to fetch email details

5.3.3 Composing Email

A key functionality for this project is the ability to compose emails from the application through voice. When the user chooses to compose email by saying “compose” the user will be given audio feedback to enter recipients address, subject and email body through voice input. Sending email is achieved by the following steps:

1. Import smtplib
2. Create the SMTP
3. Login to the server
4. Send email

The way in which this is achieved is by using Python’s smtplib module to send emails using the simple mail transfer protocol. In order to ensure messages and login credentials are not easily accessed by others, the SMTP connection must be encrypted. This can be done by the use of two protocols: SSL (Secure Sockets Layer) and TLS (Transport Layer Security). First an SMTP object is created to be used for the connection to the server. This encapsulates an SMTP connection which allows access to its methods. Since Google’s SMTP server is used to deliver emails, information such as the server, port number and authentication is required. In this case Google’s outgoing server requires SSL or TLS and is ‘smtp.gmail.com’ with port number of 587 that uses authentication. As shown in figure X. The function `starttls()` is then used to turn an existing insecure connection into a secure one by the use of encryption.

```
s = smtplib.SMTP('smtp.gmail.com', 587)
s.starttls()
```

Figure 17 : Code to establish an SMTP connection

For the application to handle more complex emails such as those that contain attachments Python’s built in email package is used. The application prompts the user to enter the recipients email address, subject and body. Multipurpose Internet Mail Extensions (MIME) is currently the most common type of email. MIME messages are handled by Python’s `email.mime` module. The `MIMEMultipart()` function sets up the MIME and the function `msg.attach(MIMEText(body, 'plain'))` then converts the body of the email to a MIME compatible string and then attaches it to the main message.

To perform the operation of sending the mail the `.sendmail()` is used once a secure connection with the server is established. The function `.quit()` terminates the SMTP session and closes the connection. An example of this is shown below:

```
s.login(user.email, user.password)
s.sendmail(user.email, newtoaddr, msg.as_string())
s.quit()
```

Figure 18 : Code to send mail

If successful the user will receive audio feedback of its successful attempt to send email. Whereas, if unsuccessful the user will be instructed to try again.

5.4 Deployment

This section highlights the technologies used to deploy the application to the server and the way in which successful deployment was achieved.

5.4.1 Heroku

Heroku is a cloud platform that enables developers to build and run applications. This platform was used to deploy and scale the application. Heroku uses Git as the primary means for deploying the application. The steps taken to deploy the application is illustrated by the diagram below:

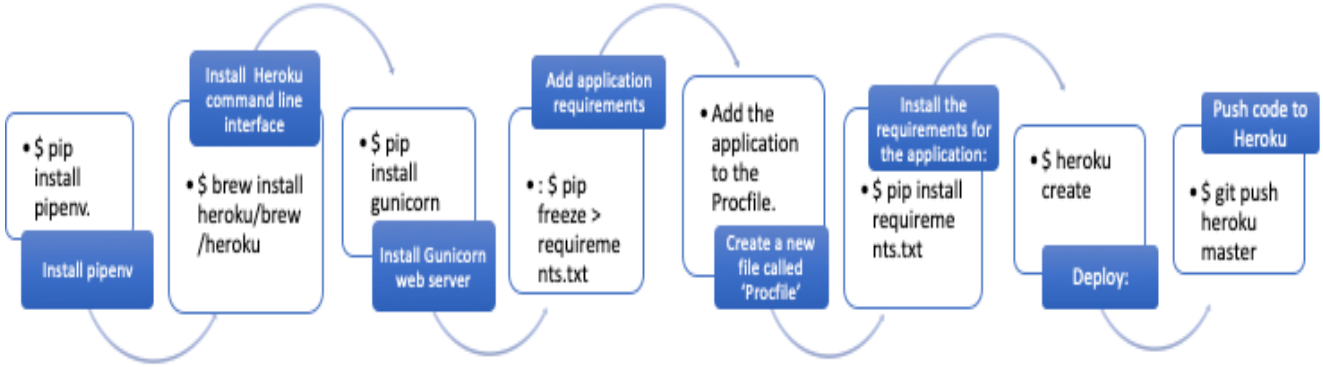

Figure 19: Steps taken to deploy application to Heroku

1. Install pipenv : `$ pip install pipenv.`
2. Install the Heroku command line interface: `$ brew install heroku/brew/heroku`
3. Install a web server called Gunicorn within the application: `$ pip install gunicorn`
4. Add the application requirements: `$ pip freeze > requirements.txt`
5. Create a new file called 'Procfile' and add the application to the Procfile.
6. Install the requirements for the application: `$ pip install requirements.txt`
7. Deploy: `$ heroku create`
8. Push code to Heroku: `$ git push heroku master`

The deployed application can be accessed via the QR code in Appendix B or by the following link: https://still-earth-64920.herokuapp.com/

5.4.2 Circle Ci

Circle Ci was used during deployment to ensure continuous integration. Every time code is pushed to the master branch on Git, Circle Ci then downloads all dependencies and pushes the latest code to the server. This is more efficient and saves time as manually deploying through terminal for every change is not required.

5.4.3 Cloudinary

Cloudinary is a cloud based storage service. It is used to securely upload and store images and videos as well as any other file from the browser to any source on the server. The service supports delivering uploaded files as attachments using their original file names. This is implemented in the application for users to add attachments of any format to emails with the original file name.

Chapter 6: **Testing**

Testing is a key part of software development as it leads to higher application quality and less bugs. This chapter with the support of test case snippets, highlights the testing strategies used and employed to rigorously assess and validate the usability and functionality of the application,

6.1 Authentication Testing

Test Case	Steps	Expected Results	Actual Results
User logging in with existing Gmail account through audio input.	1. Double click the login page to get audio output. 2. Enter email address [redacted] as speech input. 3. Enter 'abc*2019' as speech input for the password. 4. Application will verify if the correct email and password is correct if correct user will say "yes" to continue or 'no' to enter again. 5. Say "yes". 6. The application will feedback successful.	Application will feedback "Congratulations, you have logged in successfully"	Pass
Logging in with incorrect credentials through audio input	1. Double click the login page to get audio output. 2. Enter email address [redacted] as speech input. 3. Enter "test2" as the password 4. Application will verify if the correct email and password is correct if correct user will say "yes" to continue or 'no' to enter again. 5. Say "yes". 6. The application will feedback unsuccessful	Application will feedback "Invalid Login Details. Please try again."	Pass
Logging in with an email account that is not Gmail	1. Double click the login page to get audio output. 2. Enter email address [redacted], as speech input. 3. Enter "test2" as the password	Application will feedback "Invalid Login Details. Please try again."	Pass

	4. Application will verify if the correct email and password is correct if correct user will say “yes” to continue or ‘no’ to enter again. 5. Say “yes”. 6. The application will feedback unsuccessful		
Logging in with correct credentials by typing	1. Go to the login page 2. Enter [redacted] in the email field. 3. Enter “abc*2019” in the password field. 4. Click the “sign in” button	User is able to log in successfully	Pass
Logging in with incorrect credentials by typing	1. Go to the login page 2. Enter [redacted] in the email field. 3. Enter “abc” in the passwprd field. 4. Click the “sign in” button	User is not able to login	Pass
User logging out with voice when logged in.	1. Once logged in say “logout” when presented with options. 2. Application will log the user out.	Application will feedback “you have now been logged out”	Pass

Table 7: Results of Authentication Testing

6.2 Django unit testing

Testing an application as a whole is a difficult task and so dividing application into simpler modules and submodules is easier and will produce more accurate results. Django unit testing tests sections of code independently of other tests and is easy and simple to run. It helps fix bugs that are produced by latest changes.

The python’s standard library module unittest defines test using a class based approach. It provides a rich set of tools for constructing and running tests. It supports test automation, sharing of setup and shutdown code for tests into collections, and independence of the tests from the framework (Python, 2020). The following unit tests were conducted:

6.2.1 Testing URL's

Django's principal role is to decide what to do when a user asks for a particular URL. URL testing was done to ensure a Http request comes in for a particular URL and returns a http response. An example of testing a specific URL is illustrated below:

```
class TestUrls(SimpleTestCase):
      def test_compose_inbox_url_is_resolved(self):
            url = reverse('AudioMail:compose-inbox')
            self.assertEquals(resolve(url).func, compose_inbox_view)
```

Figure 20 : Test case to test URL's

After implementing a new URL, a test was done on each of them to ensure there were no errors. In order to determine if the test ran was successful the output in the terminal will display "OK". The following is the output from the running the test above:

```
System check identified no issues (0 silenced).
......
----------------------------------------------------------------------
Ran 1 tests in 0.004s
OK
```

Figure 21: Output of test case

This indicates the test has passed, thus no errors. The same outcome was achieved on all the test cases conducted. If the test was to fail the output would show an error with the type of error and the cause of it.

6.2.2 Test Case: Logging in

This test case checks if the login page loads and once logged in with either voice or keyboard input redirects to another page. In this case the inbox page. It also verifies that the IMAP connection is successful with the user's login details. The outcome of this test was a Pass.

```
        def test_login_loads(self):
            client = Client()
            response = client.get('/login/')
            self.assertEqual(response.status_code, 200)

        def test_login(self):
            u='speakmail2019@gmail.com'
            p='abc*2019'
            client = Client()
            UserDetails.objects.create(email=u, password=p)
            response = client.post('/login/', {'email' : u,
'password':p})
            self.assertEqual(response.status_code, 302)
```

Figure 22 : Test case to Log in

6.2.3 Test Case: Receiving Email

This test case ensures emails are being received. The function `def setup()` is run before the test to ensure the user is logged in before the test is run. Django's test client is then used to establish that the correct template is being rendered and emails are being received. The outcome of this test was a Pass.

```
class RecieveTestCase(TestCase):
      def setUp(self):
            u=[redacted]
            p='abc*2019'
            client = Client()
            self.client.post('/login/', {'email': u, 'password': p})

      def test_read(self):
            client = Client()
            response = self.client.post('/inbox_emails/')
            self.assertEqual(response.status_code, 200)
```

Figure 23 : Test case to receive emails

6.2.4 Test Case: Sending Emails

This test case ensures emails are being sent using the IMAP library with voice input as well as keyboard input. Before the test is run the user is logged in. Once logged in an email gets sent to the recipient's email address with the inputted subject and body. To confirm the test is successful an email was sent t[redacted]' mailbox. Thus, the outcome of the test was a pass.

```
class SendTestCase(TestCase):

      def setUp(self):
            u=[redacted]
            p='abc*2019'
            client = Client()
            self.client.post('/login/', {'email': u, 'password': p})

      def test_compose(self):
            client = Client()
            to = [redacted]
            subject = 'Testing Subject'
            body = 'Testing sending email'
            response = self.client.post('/compose-inbox/', {'tags':[to],
'subjectEmail': subject, 'emailBody': body})
            print(response.content)
```

Figure 24 : Test case to send emails

6.2.5 Summary of Unit Testing of Functionalities

Below is a table summarising the core functionalities that was tested using Django's unittest module. Including the expected result, the method and the outcome of the test. As highlighted in Table X all test cases passed.

	Feature	Expected Result	Method	Result
1.	Send email	Email is sent to recipient.	POST	PASS
2.	Receive email	Email is received in mailbox.	GET	PASS
3.	Read email	Email is read with voice output.	POST	PASS
4.	Delete email	Email is deleted from mailbox.	DELETE	PASS
5.	Search email	Email can be found when searched.	POST	PASS
6.	Mark as spam	Email is marked as spam and sent to spam folder.	POST	PASS
7.	Star/ not star email	Email is marked starred or not starred.	POST	PASS
8.	Mark email as read/unread	Email is marked as read or unread.	POST	PASS
9.	Get inbox page	Inbox page is retrieved when user is on traditional mode.	GET	PASS
10.	Creating a user object	User object is created.	GET	PASS

Table 8 : Summary of unit testing of key functionalities

6.3 Browser Compatibility Testing

Cross browser testing was done to make sure the web application works across multiple browsers. The purpose of this is to ensure that all users no matter what browser, device, or tools can use the application. The application was tested in four different browsers; namely, Google Chrome, Mozilla Firefox, Safari and Internet Explorer. Due to the limited browser support for the Web Speech API, the browsers that it currently supports are Chrome and Firefox. Thus, speech input is not supported by other browsers. The results from this test are shown below:

Browser	Feature	Result	Feature that Failed
Chrome	All features	Pass	N/A
Mozilla Firefox	All features	Pass	N/A
Safari	Most features	Pass	Voice input
Internet Explorer	Most features	Pass	Voice input
Microsoft Edge	All features	Pass	N/A

Table 9 : Results from Browser Compatibility Testing

6.4 User Acceptance Testing

The main purpose of user acceptance testing is to evaluate the end to end flow of the application against the fundamental requirements. The testing environment consists of the production like environment. The functionality is verified to confirm the application meets the specified acceptance criteria.

All functionalities were thoroughly tested by being in the position of an end user taking into consideration the user experience of those with visual impairment. The core functionalities demonstrated in Table 8 were tested manually and most functionalities performed as expected. A key issue identified was that when the application reads emails, it does not allow the whole email body to be read, instead it gets cut off. Because the project adopted an incremental method of implementation, this was quickly and successfully rectified by allowing the audio output to move onto the next message when all the content of the current message is read. Thus, this process verifies that the all functionalities work for the end user.

Chapter 7: **Evaluation**

This chapter describes the evaluation method used to measure the effectiveness of the application. Moreover, it provides an analysis of the results obtained and summarises the key findings.

7.1 Usability Evaluation

The purpose of this evaluation was to analyse how well users can learn and use the application in order to achieve the aims and goals of this project. In addition, measure the satisfaction of users whilst doing so.

Usability is made up of a combination of factors, all of which was considered in order to assess the effectiveness of the application. By taking into consideration these factors, a survey was conducted to gain feedback and measure the success of the application. The table below demonstrates the factors that were taken into account along with the questions asked to the participants after using the application.

Factor	Description	Question asked
Intuitive design	The ease of navigating through the application and understanding its architecture.	Did you find it difficult to navigate your way through the application?
Ease of Learning	How fast users can complete a task without prior knowledge of the application beforehand	What features of the application have you used? And how did you find the process?
Efficiency of Use	The speed at which users can accomplish tasks	What is the best thing about the application?
Error frequency and severity	How often users come across errors when using the system as well as how severe are these errors	Did you experience any problems logging in with voice input? If yes please indicate the issue.
Improvements	Functionalities which could be improved.	What can be improved in the application?
Subjective satisfaction	If users like the system and would recommend it.	Would you recommend this application? Do you think this application is beneficial for visually impaired users?

Table 10: Factors and questions asked when evaluating usability

7.2 Analysis of Results

10 individuals were asked to test the application and provide feedback to the questions asked in section 7.1. Moreover, senior lecturer Tony Stockman (visually impaired user) interviewed in section 3.1 was asked for general feedback of the user interface.
Below is a summary of the results:

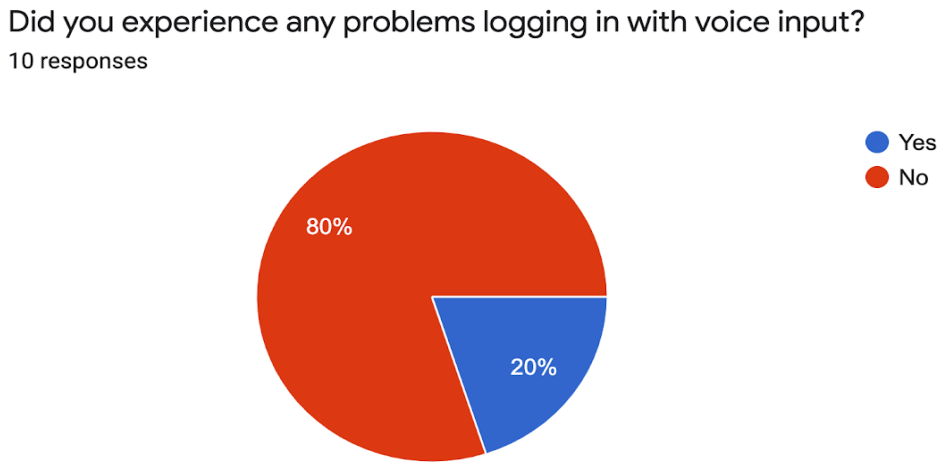

Figure 25: Pie chart displaying percentage of users who had issues logging in

As can be seen in Figure 25, 80% of the individuals asked claim they experience no problems when logging in through voice. Whereas 20% claim they did have problems. When asked what exactly the problems were the main response was that “the application did not recognise the username at the first attempt”. With speech recognition in general this is a common issue as there are many factors which affect the accuracy of the speech to text conversion. The most common being the fact that there are many words in the English dictionary which sound similar. Thus, effect the accuracy.

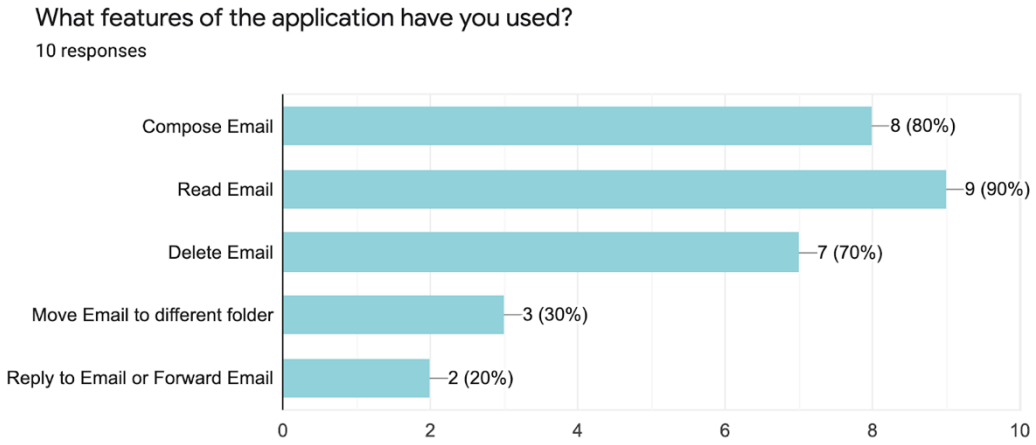

Figure 26 : Bar chart displaying the features most used

According to Figure 26, the most common feature used was reading emails, when asked how users found the process of the feature used the most common answers were that “the process was easy and simple” and “very informative”.

This demonstrates that users were able to complete a task easily without any issues.

Figure 27 shows all users feel the application is beneficial to visually impaired users. Additionally, when asked if they would recommend the application all users answered with “yes”. This portrays a successful user satisfaction and implies the aims and goal of the project was achieved.

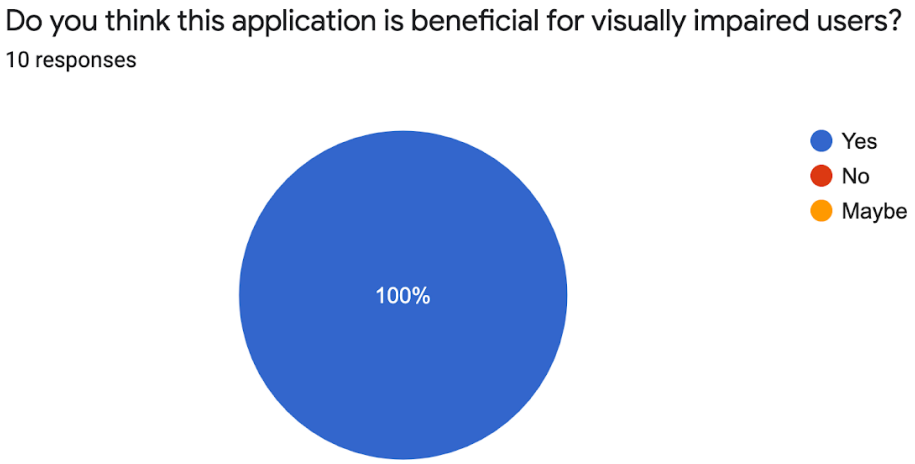

Figure 27 : Pie chart showing percentage of users who believe the application is beneficial

7.3 Summary of Evaluation

According to the results received from the survey, the main issue users had with the application was to do with the accuracy of the speech recognition which is an issue with speech recognition as a whole. This is due to computers not being on par with humans in understanding the contextual relations of words and sentences thus causing slight delays and inaccuracies with words. To overcome this issue the artificial intelligence and understanding behind voice recognition must become more sophisticated in order to prevent the misinterpretations of words.

Additionally, the following was the feedback received from Tony:

“The front page of audio mail worked fine with two different screen readers, Jaws and System Access, using Chrome, all links and text were spoken fine and there was no problem navigating the site with either screen reader.”

This proves the application to be easily accessible by visually impaired users. Hence beneficial to the target audience. Moreover, most of the feedback received was positive. Thus, it can be concluded that the application is well suited for its intended purpose. With the most popular feature being the fact that it is multimodal and can cater for the needs of all users hence make reading and sending emails much easier for the visually impaired.

Chapter 8: **Conclusions**

This chapter concludes my views on the project, the things that went well, the challenges faced, the lessons I have learnt and what I would do if I had more time.

8.1 Achievements

Overall the project has proven to be a success and a huge milestone was reached. The aims and objectives of the project highlighted in chapter 1 outlined a distinct opportunity for the development of the application. The background research conducted in chapter 2 indicated the way in which my application would differ from similar systems and made it possible to gather precise requirements for the implementation of the application. The functional, non-functional and security requirements stated in chapter 3 have all successfully been met and thoroughly tested throughout while taking into consideration the risks outlined in Appendix C. Furthermore, the application was successfully deployed to the server.

Moreover, working with voice recognition technologies was a technical challenge within itself in which I can say I have effectively overcome. The fundamental aim of the project was to provide visually impaired users an interactive email system in which they can send and receive emails with voice along with other email functionalities. Not only was this aim achieved, the implementation went beyond the scope of the project to make it accessible to all users by implementing a multimodal system where users can switch between the voice mode and traditional mode.

Additionally, I have learnt many lessons a key lesson being the fact that planning is essential for the success of a project as well as continuous testing to ensure the application is functional. Prior to the project, I had limited knowledge and confidence in front end programming languages. I have now become more confident in the Django framework, Python, JavaScript, HTML and CSS and have enhanced my programming ability.

8.2 Challenges

Throughout the project I have come across many challenges. The most difficult challenge was trying to deploy the application to the server. When I initially deployed my application, the libraries were not working properly. Audio feedback was not being played and functioning as planned. This was a major issue as it was a core functionality in my project. The initial libraries used were python's speechrecognition library for speech to text and Google text to speech. Both backend libraries. To ensure successful deployment with all features and functionalities working I had to find a front-end replacement for the backend libraries in order for speech recognition and speech synthesis to work on the server.

Moreover, another key issue faced was the accuracy of the voice recognition. There were times when the recognition system misinterpreted words from

speech input. This is an issue for all voice recognition systems in general as many words sound very similar and cannot be avoided.

Furthermore, a non-technical challenge I have faced was to do with time management. Balancing working on the project along with other third year modules, interviews and applications for graduate jobs, as well as working part time was quite overwhelming. It was difficult to prioritise everything. However, by splitting up the work into smaller chunks and balancing the time spent in working on the project and other responsibilities I was able to implement all the project's functionalities.

Moreover, as highlighted in Appendix A, due to circumstances beyond anyone's control, education around the world was highly impacted. It was challenging to continue working on the project whilst witnessing severe disruption worldwide. Nevertheless, with perseverance and dedication I managed to build up the motivation to complete the project to the best of my abilities.

8.3 Future Improvements

As with the majority of software development projects there is several room for improvement. In my case based on user feedback a key aspect I would implement in my application is the ability for users to log in with any email account as currently users are only limited to Gmail. This will allow more users particularly those who use other email accounts as their main form of contact as well as companies that use enterprise email accounts to make use of the application functionalities.

Additionally, giving users the option to choose any language in which they would like the interactive voice response to operate will cater for a more global audience and make translating text easier hence, improve communication.

Furthermore, extra features and functionalities could be added to improve user experience and enhance system security such as the ability to send and accept calendar invites via voice and the ability to enable email filtering to automatically detect spam emails with natural language processing technologies.

8.4 Overall Conclusion

Overall, the successful completion of this project has proven that an application that sends and receives emails without the use of a keyboard can be achieved and be beneficial to those who are visually impaired. Additionally, not only is the application useful for visually impaired users it is also beneficial to all users as it adopts a multi modal functionality. Furthermore, through background research, I have become more aware of the importance of accessibility and how it is crucial to consider accessibility when developing applications in order to provide equal access and opportunity to everyone.

Moreover, plenty of obstacles were overcome throughout the journey of this project and milestones were achieved. With all the key functionalities implemented as well as going beyond the set requirements of the application to make the system multimodal. I can conclude that this project is a success.

References

Broida, R. (2019). *Dictate email messages in Microsoft Outlook*. [online] CNET. Available at: https://www.cnet.com/how-to/dictate-email-messages-in-microsoft-outlook/

Djangoproject.com. (2019). *Download Django | Django*. [online] Available at: https://www.djangoproject.com/download/

Docs.microsoft.com. (2019). *How to use Outlook REST APIs in a Python app - Outlook Developer*. [online] Available at: https://docs.microsoft.com/en-us/outlook/rest/python-tutorial#designing-the-app

Guru99.com. (2019). [online] Available at: https://www.guru99.com/introduction-webdriver-comparison-selenium-rc.html

Hackernoon.com. (2019). *Advantages and Disadvantages of Django*. [online] Available at: https://hackernoon.com/advantages-and-disadvantages-of-django-499b1e20a2c5

Humanising Technology Blog. (2019). *What is a screen reader?*. [online] Available at: https://www.nomensa.com/blog/2005/what-screen-reader

Mistry, J. and Posts, S. (2019). *The Ultimate Guide to Accessible Emails - Litmus*. [online] Litmus Software, Inc. Available at: https://litmus.com/blog/ultimate-guide-accessible-emails

Prater, D., Smart, C., Griffith, D., Prater, D., Homme, J., Brain, V., Prater, D., Prater, D., Wocher, G. and Matthews, T. (2019). *Why I'm still on a Mac: a guide for skeptics and those who feel that grass may be greener with Windows | AppleVis*. [online] Applevis.com. Available at: https://www.applevis.com/guides/why-im-still-mac-guide-skeptics-and-those-who-feel-grass-may-be-greener-windows

RNIB - See differently. (2019). *Top screen readers for Windows*. [online] Available at: https://www.rnib.org.uk/rnibconnect/screen-reader-windows-review

Stripo.email. (2019). *Email Accessibility Guidelines: Standards & Best Practices — Stripo.email*. [online] Available at: https://stripo.email/blog/email-accessibility-guidelines-standards-best-practices/

Tutorialspoint, 2020. *Python - Sending Email Using SMTP - Tutorialspoint*. [online] Tutorialspoint.com. Available at: <https://www.tutorialspoint.com/python/python_sending_email.htm> [Accessed 9 May 2020].

Understanding Transcription. (2019). *Voice Recognition Software: Pros and Cons - Understanding Transcription*. [https://www.voxtab.com/transcription-blog/voice-recognition-software-pros-and-cons/

Appendix

Appendix A: Unforeseen Circumstances

From Monday 23rd March 2020, the United Kingdom has gone into temporary lockdown. The government has closed schools, pubs, restaurants, cafes, gyms and other businesses under new lockdown measures to fight against the coronavirus outbreak. The coronavirus is a new illness that can affect the lungs and airways and is very infectious. The World Health Organisation declared the outbreak to be a Public Health Emergency of International concern and from 11th March 2020 recognised this virus as a pandemic. This pandemic has led to severe global socioeconomic disruption and widespread fears. In particular students across the globe are faced with disruptions to their education. Following government advice as universities are shut with everyone adhering to the social distancing rules, and with many individuals anxious and facing challenging circumstances, it is therefore difficult to gain accurate user feedback within the deadline of this project. Thus, the quality of user feedback and evaluation may have been effected

Appendix B: QR code for application

Appendix C: Risk Analysis

Description of Risk	Description of Impact	Likelihood Rating	Impact Rating	Prevention Actions
1. The Speech to text conversion is not very accurate	The Voice input is not accurate when converted to speech	Medium	High	Try different libraries in python and select the best text to speech conversion
2. The text to speech conversion not being very accurate	Incorrected words being outputted by the application.	Medium	High	Do thorough testing of text to speech to ensure there is no errors in particular words.
3. Voice commands not working e.g. "compose email"	Application will not work with basic voice commands	Medium	High	Have a reliable microphone and ensure basic functionality works.
4. Text to speech or speech to text not working at all.	The main goal of the project would not be achieved.	Medium	High	Ensure the most accurate python library is installed.
5. Security Concerns e.g. password input	Low security is a huge privacy concern for users.	Medium	High	Enable a secure way to input password using other authentication methods.
6. Work loss	Could lead to complications in the project	Low	High	Consistently save work and keep back up of work on the cloud and in different locations
7. Connection errors during the login stage	Not being able to log into the application through voice input and a security concern	Medium	High	Ensure the right credentials are entered each time.
8. Not enough testing	Could lead to a failed project. Main functionalities	Low	Medium	Ensure there is at least 2-3 weeks set aside

	may not be working.			for testing and fixing errors
9. Time constraints	Neglecting the project due to focusing more on exams	Medium	High	Manage time efficiently. By setting aside a decent amount of time to work on the project.
10. Illness	May cause delays in continuing to work on my project	Medium	Medium	Inform supervisor and school if illness is severe. Catch up with tasks after recovery.
11. Unforeseen Circumstances	Things out of my control could cause delays in the project.	Medium	Medium	Inform supervisor and school immediately and figure out what can be done.